ScriptWell — Online Book Store

Mini Project Report · Advanced Internet Programming & PL/SQL

Java Servlets · JSP · JDBC
Oracle Database Free 23ai · PL/SQL
Tomcat 10.1 · JDK 17 · Eclipse IDE

MVC architecture | Jakarta Servlets 6.0 | JSP + JSTL | Bootstrap 5 | JavaScript (Fetch API) | CallableStatement | Packages | Triggers | Cursors | Exception handling

1. Introduction and Objectives

ScriptWell is a responsive e-commerce web application for buying books online. Customers can browse a catalogue of 34 books in 7 categories, search and filter them, manage a shopping cart, buy instantly with *Buy Now*, pay through a simulated Card/UPI/Cash-on-Delivery gateway and track or cancel their orders. Administrators get a dashboard for book CRUD, inventory and order management. The project has two aims. For **Advanced Internet Programming**, it applies the MVC pattern with Servlets, JSP, filters, sessions and JDBC. For **PL/SQL**, it places the core business rules in the database as packages, procedures, functions, triggers, cursors and custom exceptions. Java calls this logic only through `CallableStatement`.

2. System Architecture



Figure 1 – Three-tier MVC architecture. Servlets never contain SQL. They call DAO methods, which run stored PL/SQL through JDBC `CallableStatement` and read results from `SYS_REFCURSOR` OUT parameters.

3. Database Design

Table	Key columns / constraints
CATEGORIES	category_id (PK, identity), name UNIQUE
BOOKS	book_id PK, category_id FK, isbn UNIQUE, CHECK price>0, stock≥0, rating 0–5
USERS	user_id PK, email UNIQUE, password_hash (SHA-256), role CHECK IN (CUSTOMER, ADMIN)
CART_ITEMS	UNIQUE(user_id, book_id), CHECK qty 1–99, ON DELETE CASCADE
ORDERS	order_id PK (from 1001), status CHECK, payment_method CHECK
ORDER_ITEMS	order_id FK, book_id FK, unit_price (price at time of sale)
INVENTORY_LOG	old/new stock, change, reason (written by trigger)
ORDER_STATUS_AUDIT	old/new status history (written by trigger)

The view `V_BOOK_CATALOG` joins books with categories and is used by every catalogue query. Indexes on `category_id`, `UPPER(title)` and `orders.user_id` speed up search and order history.

4. PL/SQL Implementation

Object	Responsibility
`pkg_user`	register, login (`STANDARD_HASH`), list users
`pkg_book`	search with filters and sorting (REF CURSOR), add/update/delete, adjust stock
`pkg_cart`	MERGE-based add, update, remove, `cart_total`, `cart_count`
`pkg_order`	`place_order`, `buy_now`, history, status update, cancel with restock
Functions	`fn_shipping_charge`, `fn_is_in_stock`, `fn_total_revenue`
Procedure	`sp_get_dashboard_stats` (6 OUT parameters)
Triggers	email normalisation, book validation, stock audit, order status audit

Checkout transaction: `place_order` opens an explicit cursor over the cart with `FOR UPDATE`, which locks the book rows. For each line it checks stock, inserts the order line and decrements stock, then clears the cart and `COMMIT`s. If anything fails, the exception handler closes the cursor and rolls back the whole order. Business errors use `RAISE_APPLICATION_ERROR(-20001…-20010)`, and Java turns them into friendly messages, e.g. *"Only 1 copy of 'The Psychology of Money' available"*.

5. Web Implementation Highlights

- **MVC:** JavaBeans (model), JSP views under `WEB-INF` (not directly reachable), Servlets as controllers using `@WebServlet`.
- **Session management:** the logged-in user and cart badge count are kept in `HttpSession`. The session ID is regenerated at login to prevent session fixation.
- **Security:** `AuthFilter` protects cart, checkout, orders and `/admin/*` (role check, HTTP 403). Output is escaped with `fn:escapeXml`, and all SQL uses bind parameters.
- **Reusable UI:** header/footer fragments and a custom JSP tag file `<sw:bookCard>`.
- **JavaScript:** AJAX add-to-cart (Fetch API + JSON) with toast messages, quantity steppers, card formatting and Luhn validation, and a payment-processing animation. Chart.js draws the admin sales chart.
- **Responsive design:** Bootstrap grid, custom CSS design tokens and 35 generated SVG book covers.

6. Screenshots

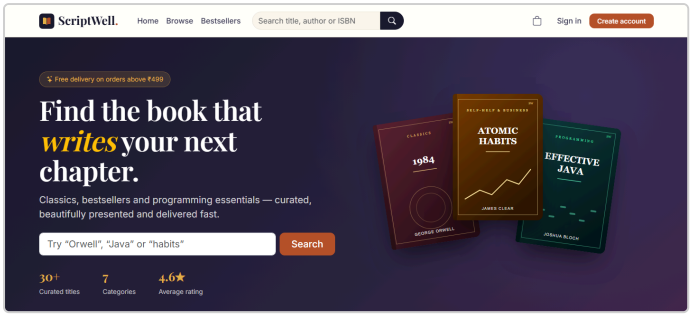


Fig. 2 – Home page with hero, search and categories

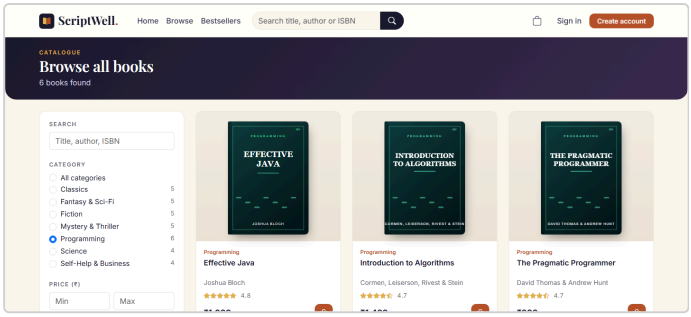


Fig. 3 – Catalogue with category, price filters and sorting

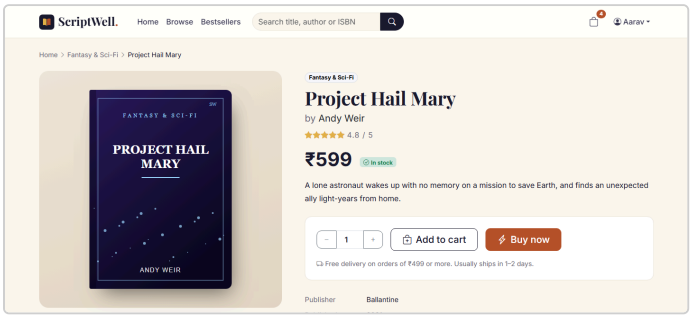


Fig. 4 – Book details with quantity, Add to cart and Buy Now

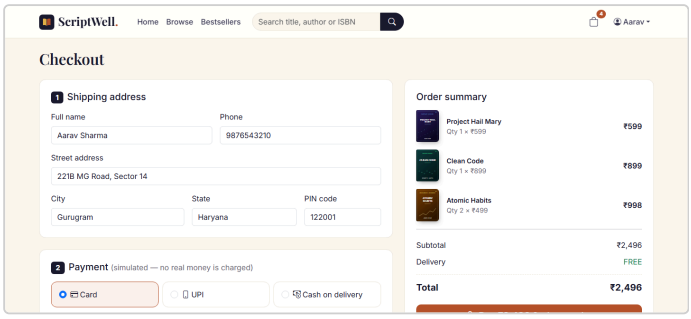


Fig. 5 – Checkout with simulated Card / UPI / COD payment

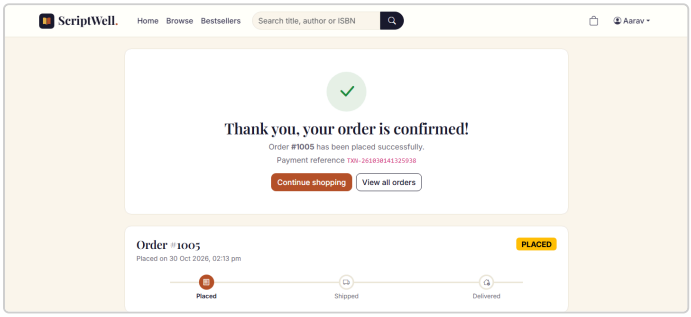


Fig. 6 – Order confirmation with payment reference and tracking

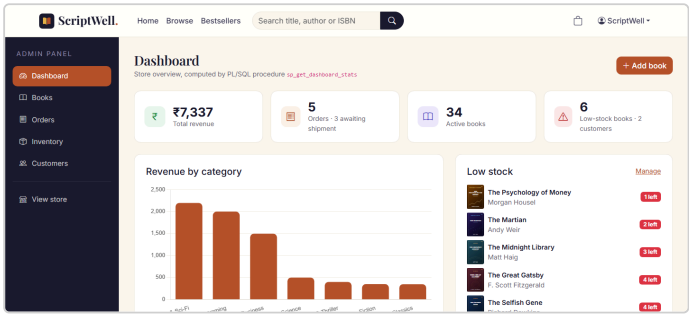


Fig. 7 – Admin dashboard (stats from `sp_get_dashboard_stats`)

7. Testing and Results

#	Test case	Expected result	Result
1	Register with an email that already exists	pkg_user raises -20001 → "An account with this email already exists"	Pass
2	Login with wrong password / correct password	"Invalid email or password" / session created, redirect (admin → dashboard)	Pass
3	Search "java", category filter, price range ₹400–699, sorting	2, 6 and 16 matching books returned in the chosen order	Pass
4	Add 5 copies of a book that has 1 in stock	-20003 "Only 1 copy … available". Cart unchanged	Pass
5	Checkout with invalid card number (fails Luhn)	Rejected before calling the database, form shown again with error	Pass
6	Place order from cart / Buy Now	Order created, stock reduced, cart cleared, INVENTORY_LOG rows added by trigger	Pass
7	Customer opens another user's order or `/admin`	HTTP 404 (owner check in `get_order`) / HTTP 403 from AuthFilter	Pass
8	Cancel a PLACED order; reopen a cancelled order (admin)	Stock restored by cursor loop / -20009 "A cancelled order cannot be reopened"	Pass
9	Admin adds book with price -5 or year 2099	Rejected by `pkg_book.validate_book` / trigger `trg_books_before_save`	Pass
10	Admin deletes a book that has been sold / never sold	Deactivated (order history kept) / permanently deleted	Pass

Result: all functional test cases passed on Apache Tomcat 10.1 with Oracle Database Free 23ai. The PL/SQL scripts compile with no errors (`USER_ERRORS` is empty). The application is responsive on desktop and mobile widths.

8. Conclusion and Future Scope

ScriptWell meets the project objectives. It is a complete, good-looking online bookstore built with Servlets, JSP and JDBC in a clean MVC structure. Its core rules (stock control, pricing, checkout transactions, auditing and validation) are enforced in Oracle PL/SQL, so they apply no matter which client calls them. Keeping these rules in the database made the Java code simpler and kept the data consistent even when several users order at the same time. Future improvements could add a real payment gateway, e-mail notifications, wish lists and reviews, connection pooling through a JNDI DataSource, and a REST API for a mobile app.